

Aula 5.5

Seleção de features

Filtros, wrappers e embedded

DATASET	PRÁTICA	DURAÇÃO
California Housing	Reduzir colunas sem perder score	2 horas

Ao fim da aula, você vai conseguir

- 1 Classificar qualquer método de seleção como filtro, wrapper ou embedded.
- 2 Ranquear features com ANOVA F e informação mútua e explicar por que os rankings divergem.
- 3 Rodar RFE, RFECV e SequentialFeatureSelector e estimar quantos ajustes cada um custa.
- 4 Usar Lasso e Random Forest (MDI e permutação) como seletores via SelectFromModel.
- 5 Colocar a seleção dentro do Pipeline, para que o score da validação cruzada seja honesto.

Da criação de features à escolha de features

AULA 5.4 · O QUE FICOU

Min-Max, Standard e Robust reagem de forma diferente ao mesmo outlier.

One-hot, ordinal e target encoding, cada um com seu risco.

Cada feature derivada teve o ganho medido na métrica.

AULA 5.5 · A PERGUNTA DE HOJE

Depois de criar dezenas de colunas, quais delas o modelo realmente precisa?

Critério da prática: menos colunas, mesmo R^2 na validação cruzada.

03

Teoria

O conceito com a matemática visível, antes do código.

Por que remover colunas

Generalização	Custo	Interpretação	Robustez
Ruído dá ao modelo mais chances de decorar coincidências do treino.	Menos tempo de treino e inferência, menos dados para coletar em produção.	Seis colunas cabem numa explicação para o negócio. Sessenta não cabem.	Cada coluna é uma dependência que pode quebrar ou sofrer drift.

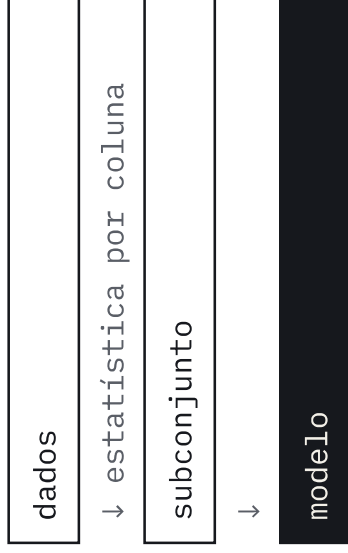
Seleção mantém um subconjunto das colunas originais. O nome e o significado de cada uma continuam intactos.

Extração (PCA, aula 6.11) cria colunas novas combinando as antigas. Reduz dimensão, mas perde o nome.

As três famílias de seleção

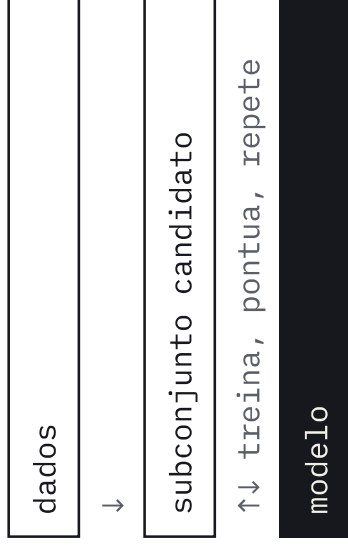
O critério é quando o modelo participa da escolha.

FILTRO



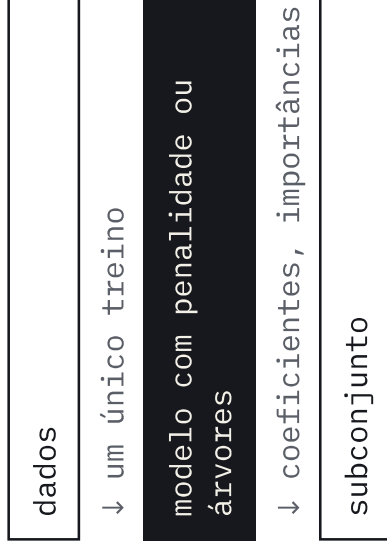
O modelo não participa. Variância, correlação, ANOVA F, informação mútua.

WRAPPER



O modelo avalia cada subconjunto. RFE, RFECV, forward e backward.

EMBEDEDDED



A seleção sai do próprio treino. Lasso, Random Forest, boosting.

O dataset: California Housing

COLUNA	SIGNIFICADO	R COM ALVO
MedInc	Renda mediana	0,69
HouseAge	Idade mediana das casas	0,11
AveRooms	Cômodos por domicílio	0,15
AveBedrms	Quartos por domicílio	-0,05
Population	População do bloco	-0,02
AveOccup	Moradores por domicílio	-0,02
Latitude	Posição norte-sul	-0,14
Longitude	Posição leste-oeste	-0,05

20.640

linhas, sem valores ausentes

MedHouseVal

alvo, em US\$ 100 mil, com teto em 5,0

MONTAGEM DA AULA

8 colunas reais + 8 colunas de ruído gaussiano. Um bom seletor descarta as 8 de ruído.

Filtros de primeira passada: variância e correlação

Variância quase zero

Uma coluna que quase não varia não consegue separar exemplos. Remova antes de qualquer outro método.

$$\text{Var}(X) = E[(X - \mu)^2] < \text{limiar}$$

Depende da escala. Use limiar 0 no dado cru ou compare só depois de escalar.

Redundância entre colunas

Duas colunas muito correlacionadas repetem a mesma informação.

$$r(\text{AveRooms}, \text{AveBedrms}) \approx 0,85$$

Em pares com $|r|$ acima de 0,9, fique com a de significado mais claro. Coeficientes lineares ficam instáveis com as duas.

ANOVA F: quanto a feature explica o alvo linearmente

f_classif · alvo categórico

$$\frac{\sum n_g (\bar{X}_g - \bar{X})^2 / (G - 1)}{\sum \sum (x - \bar{x}_g)^2 / (N - G)}$$

Variância entre as classes dividida pela variância dentro delas. F alto: as médias por classe se afastam.

f_regression · alvo contínuo

$$F = \frac{r^2}{1 - r^2} \cdot (n - 2)$$

É a correlação de Pearson reescrita como estatística F. Só enxerga relação linear.

Com 16 mil linhas, quase tudo passa no teste de significância. Use o score para ranquear e a validação cruzada para decidir o k.

Informação mútua: qualquer tipo de dependência

$$I(X; Y) = \sum \sum p(x, y) \cdot \log \frac{p(x, y)}{p(x) p(y)}$$

Zero só na independência

Capta relações não lineares e não monotônicas que o F não vê.

Estimada por vizinhos

Para dados contínuos, o sklearn usa k-NN.
É estocástica: fixe random_state.

Só para ranquear

Não tem escala fixa nem p-valor. Compare features entre si, não entre datasets.

Latitude e Longitude nos dois rankings

Pelo F, parecem fracas

Pearson de $-0,14$ e $-0,05$. O preço não sobe nem desce de forma regular de norte a sul ou de leste a oeste.

Pela MI, sobem para o topo

O preço depende da posição de forma não monotônica: litoral e regiões metropolitanas caras, interior barato.

LIMITE DOS FILTROS UNIVARIADOS

Cada coluna é avaliada sozinha.

Latitude e Longitude juntas definem um ponto no mapa e valem mais que a soma das duas. Essa interação só aparece quando o modelo participa: wrappers e embedded.

Wrappers: busca no espaço de subconjuntos

2p

subconjuntos possíveis com p colunas

p = 8

256

p = 16

65.536

p = 30

1.073.741.824

Busca exaustiva só é viável com poucas colunas. Na prática, usa-se busca gulosa.

Forward

Começa vazio. A cada rodada, testa todas as colunas restantes e adiciona a que mais melhora o score.

Backward

Começa com todas. A cada rodada, remove a coluna cuja saída menos piora o score.

RFE

Backward barato: em vez de testar cada remoção, descarta a coluna de menor coeficiente ou importância.

RFE e RFECV, passo a passo

1

Treina o modelo com as colunas atuais.

2

Lê coef_ ou feature_importances_.

3

Remove as step colunas menos importantes.

4

Repete até restar n_features_to_select.

RFECV escolhe o número

Roda o RFE dentro de cada fold e fica com o número de colunas de maior score médio. A curva completa fica em cv_results_.

Escale antes de usar coef_

Sem escalonamento, o tamanho do coeficiente reflete a unidade da coluna. Population em pessoas e MedInc em dezenas de milhares não se comparam.

SequentialFeatureSelector: forward e backward com validação

```
from sklearn.feature_selection import (
    SequentialFeatureSelector)

sfs = SequentialFeatureSelector(
    HistGradientBoostingRegressor(),
    n_features_to_select=6,
    direction="forward", # ou "backward"
    scoring="r2",
    cv=5,
    n_jobs=-1,
)
```

Aceita qualquer modelo

Não precisa de coef_nem de importâncias. Funciona com KNN e HistGradientBoosting.

Custo do forward

Para escolher k de p colunas:

$$[p + (p-1) + \dots + (p-k+1)] \times cv$$

Lasso: a penalidade L1 zero coeficientes

$$\min_w \frac{1}{2n} \|y - Xw\|_2^2 + \alpha \|w\|_1$$

Por que zero

A região L1 é um losango com quinas nos eixos. A solução costuma cair numa quina, onde um coeficiente é exatamente zero.

α controla quantas saem

Quanto maior o α , mais coeficientes zerados. LassoCV escolhe α por validação cruzada.

Colunas correlacionadas

Fica com uma quase ao acaso e zera a outra. ElasticNet distribui o peso entre as duas.

Importância em árvores: impureza e permutação

MDI · FEATURE_IMPORTANCES_

Soma a redução de impureza de todos os cortes que usam a coluna, média nas árvores.

Grátis: vem do treino.

Favorece colunas contínuas e de alta cardinalidade. Ruído contínuo recebe importância acima de zero.

Medida no treino, inclui o que o modelo decorou.

PERMUTAÇÃO · PERMUTATION_IMPORTANCE

Embaralha uma coluna na validação e mede quanto o score cai.

Sem viés de cardinalidade. Serve para qualquer modelo.

Custa $p \times n_{\text{repeats}}$ predições.

Colunas correlacionadas se mascaram: embaralhar uma não dói se a gêmea continua lá.

Custo computacional de cada abordagem

Cenário: $p = 20$ colunas, validação cruzada com 5 folds.

FAMÍLIA	MÉTODO	CONTA	AJUSTES
Filtro	f_regression	uma passada, $O(n \cdot p)$	0
Filtro	mutual_info_regression	k-NN por coluna	0
Embedded	Lasso ou RF (MDI)	um treino	1
Embedded	LassoCV, 100 alphas	100×5 , com warm start	500
Wrapper	RFE, step = 1	$\approx p$	20
Wrapper	RFECV	$\approx p \times 5$	100
Wrapper	SFS forward até 10	$(20 + 19 + \dots + 11) \times 5$	775
Wrapper	Busca exaustiva	$2^{20} \times 5$	$\approx 5,2$ milhões

A seleção é um passo de treino e fica dentro do Pipeline

VAZA

HONESTO

```
sel = SelectKBest(f_regression, k=6)
X_sel = sel.fit_transform(X_tr, y_tr)

cross_val_score(modelo, X_sel, y_tr,
                 cv=cv)

# o seletor já viu os folds de validação
```

```
pipe = Pipeline([
    ("sel", SelectKBest(f_regression, k=6)),
    ("m", modelo),
])

cross_val_score(pipe, X_tr, y_tr, cv=cv)

# seleção refeita em cada fold
```

A seleção aprende com o alvo, como o scaler aprende média e desvio. Tudo que aprende com dados entra no Pipeline.

04

Quiz rápido

Três perguntas para checar a base antes do código.

Três perguntas

- 1** Você rodou SelectKBest no treino inteiro e depois cross_val_score nas colunas escolhidas. O R^2 reportado tende a ficar acima, abaixo ou igual ao real?
- 2** Latitude tem Pearson $-0,14$ com o preço, mas informação mútua alta. O que isso diz sobre a relação entre as duas?
- 3** Com 30 colunas e $cv=5$, quantos ajustes faz um SFS forward para escolher 3 colunas?

Respostas

- 1 Acima. O seletor viu as linhas de validação de todos os folds. Quanto mais ruído e menos linhas, maior a inflação.
- 2 A relação existe, mas não é linear nem monotônica. O preço sobe e desce ao longo da latitude conforme a região.
- 3 $(30 + 29 + 28) \times 5 = 435$ ajustes.

05

Exercício guiado

Reduzir colunas sem perder score, no California Housing.

Passo 1: dados, ruído, teste trancado e baseline

```
import numpy as np, pandas as pd, time
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.ensemble import HistGradientBoostingRegressor

X, y = fetch_california_housing(return_X_y=True, as_frame=True)
rng = np.random.default_rng(42)
for i in range(8):
    X[f"ruído_{i}"] = rng.normal(size=len(X))    # 8 colunas que devem sair

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)
cv = KFold(n_splits=5, shuffle=True, random_state=42)
modelo = HistGradientBoostingRegressor(random_state=42)

def avaliar(est):
    t = time.perf_counter()
    s = cross_val_score(est, X_tr, y_tr, cv=cv, scoring="r2", n_jobs=-1)
    return round(s.mean(), 4), round(s.std(), 4), round(time.perf_counter() - t, 1)

avaliar(modelo)

# baseline: 16 colunas
```

Passo 2: filtros ANOVA F e informação mútua

```
from functools import partial
from sklearn.pipeline import Pipeline
from sklearn.feature_selection import (
    SelectKBest, f_regression, mutual_info_regression)

mi = partial(mutual_info_regression, random_state=42)

for nome, f in [("F", f_regression), ("MI", mi)]:
    for k in [2, 4, 6, 8, 12, 16]:
        pipe = Pipeline([
            ("sel", SelectKBest(f, k=k)),
            ("m", modelo),
        ])
        print(nome, k, avaliar(pipe))
```

Observe

Em que k cada curva alcança o baseline.

Confira as colunas

Ajuste o pipe e leia
pipe[:-1].get_feature_names_out().

Pergunta

Com k = 8, algum ruído entrou? Em qual dos dois filtros?

Passo 3: wrapper com RFECV

```
from sklearn.feature_selection import RFECV
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler

pipe_rfe = Pipeline([
    ("sc", StandardScaler()),
    ("sel", RFECV(Ridge(), step=1, cv=3, scoring="r2")),
    ("m", modelo),
])

avaliar(pipe_rfe)  # validação aninhada

pipe_rfe.fit(X_tr, y_tr)
rfe = pipe_rfe.named_steps["sel"]
print(rfe.n_features_, X_tr.columns[rfe.support_].tolist())
```

Ranqueador barato

Ridge ordena as colunas. O modelo final pode ser outro.

Aninhado

cv=3 escolhe o número de colunas dentro de cada um dos 5 folds externos.

Plote

rfe.cv_results_["mean_test_score"] contra o número de colunas.

Passo 4: embedded com Lasso e Random Forest

```
from sklearn.feature_selection import SelectFromModel
from sklearn.linear_model import LassoCV
from sklearn.ensemble import RandomForestRegressor
from sklearn.inspection import permutation_importance

pipe_lasso = Pipeline([
    ("sc", StandardScaler()),
    ("sel", SelectFromModel(LassoCV(cv=3, random_state=42))), # fica quem não zerou
    ("m", modelo),
])
avaliar(pipe_lasso)

X_a, X_v, y_a, y_v = train_test_split(X_tr, y_tr, test_size=0.25, random_state=42)
rf = RandomForestRegressor(n_estimators=200, n_jobs=-1, random_state=42).fit(X_a, y_a)
perm = permutation_importance(rf, X_v, y_v, n_repeats=10, random_state=42, n_jobs=-1)
pd.DataFrame({"MDI": rf.feature_importances_, "perm": perm.importances_mean},
              index=X.columns).sort_values("perm", ascending=False)
```

Passo 5: comparar e abrir o teste uma vez

MÉTODO	COLUNAS	R ² CV	TEMPO
Baseline	16		
SelectKBest F			
SelectKBest MI			
RFECV + Ridge			
Lasso			

Critério de escolha

Menos colunas com R² dentro de um desvio padrão do baseline.

```
final = pipe_escolhido.fit(X_tr, y_tr)
final.score(X_te, y_te)
# uma única vez
```

Preenchida ao vivo com os números da turma.

06

Erros comuns

As armadilhas específicas da seleção de features.

Seis armadilhas da seleção de features

1 Selecionar fora da validação

O seletor vê os folds de validação e o score infla.

3 Confiar só no MDI

Ruído contínuo recebe importância. Confirme com permutação.

2 Coeficientes sem escalar

Lasso e RFE linear passam a ranquear pela unidade da coluna.

4 Cortar gêmeas pela importância individual

Colunas correlacionadas dividem a importância. Remover as duas derruba o score.

5 Escolher k olhando o teste

O teste vira validação e deixa de medir o mundo real.

6 Ler importância como causa

Importância diz que o modelo usa a coluna, não que ela causa o alvo.

Para ir além: quatro métodos avançados

Stability selection

Repete a seleção em muitas subamostras e mantém só as colunas escolhidas na maioria delas.

Boruta

Cria uma cópia embaralhada de cada coluna. Fica quem supera a melhor cópia de forma consistente.

mRMR

Máxima relevância com o alvo e mínima redundância entre as escolhidas, por informação mútua.

Seleção por SHAP

Média do valor absoluto de SHAP como critério de corte. Volta na aula 10.4.

Desafio: seis colunas, mesmo score

Parta das 16 colunas da aula (8 reais e 8 de ruído) mais as features derivadas que você criou na 5.4.

Compare um método de cada família e entregue o melhor pipeline com no máximo 6 colunas.

Entrega: notebook no Colab, corrigido no início da aula 5.6.

CRITÉRIO DE ACEITE

Seleção dentro do Pipeline, validada com KFold de 5 folds e semente 42.

No máximo 6 colunas, com R^2 médio no máximo 0,01 abaixo do baseline.

Tabela com colunas, R^2 , desvio e tempo para filtro, wrapper e embedded.

Nenhuma coluna de ruído no conjunto final.

Teste aberto uma única vez, na última célula.

Documentação oficial do scikit-learn

Guia do usuário 1.13 · Feature selection

scikit-learn.org/stable/modules/feature_selection.html

Guia do usuário 4.2 · Permutation importance

scikit-learn.org/stable/modules/permutation_importance.html

`f_regression` e `mutual_info_regression`

[API · sklearn.feature_selection](#)

RFE, RFECV e SequentialFeatureSelector

[API · sklearn.feature_selection](#)

SelectFromModel e LassoCV

[API · feature_selection · linear_model](#)

`fetch_california_housing`

[API · sklearn.datasets](#)

Próxima aula

5.6 · Laboratório 1: Titanic de ponta a ponta